

Mirror Descent and Duality

jasonxiong789

December 2024

1 Duality

Given the primal problem:

$$\begin{aligned} \min_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

the dual can be written as:

$$\begin{aligned} \max_{\lambda, \nu} \quad & \min_{x \in \mathcal{D}} \left(f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{j=1}^p \nu_j h_j(x) \right) \\ \text{subject to} \quad & \lambda \geq 0 \end{aligned}$$

Note that we typically denote the Lagrangian L as

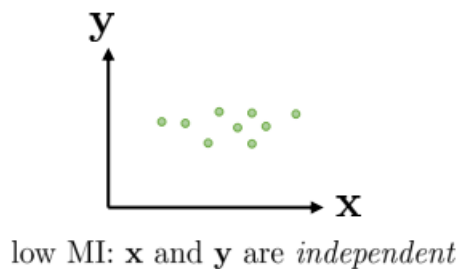
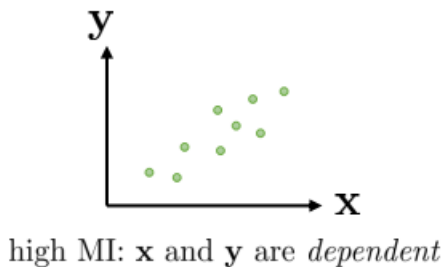
$$L(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{j=1}^p \nu_j h_j(x)$$

and the dual function g as

$$\min_{x \in \mathcal{D}} L(x, \lambda, \nu)$$

1a. Duality Geometric Interpretation

$$\mathcal{G} = \{(f_1(x), \dots, f_m(x), h_1(x), \dots, h_p(x), f_0(x)) \in \mathbb{R}^m \times \mathbb{R}^p \times \mathbb{R} \mid x \in \mathcal{D}\}$$



For a fixed λ , the Lagrangian for any given point $\tilde{x} \in \mathcal{D}$ that is associated with the point $(\tilde{u} = f_1(\tilde{x}), \tilde{t} = f_0(\tilde{x})) \in \mathcal{G}$ can be written as

$$L(\lambda, \tilde{x}) = \lambda \tilde{u} + \tilde{t}$$

$$\implies \tilde{t} = -\lambda \tilde{u} + L(\lambda, \tilde{x})$$

This is essentially, saying the Lagrangian for the point $(\tilde{u}, \tilde{t}) \in \mathcal{G}$ is the value of the t -intercept if we drew a line with $-\lambda$ that crosses the point $(\tilde{u}, \tilde{t})$. Thus, the value of the dual function g is the smallest possible t -intercept over all lines with slope $-\lambda$ that goes through at least 1 point in $\mathcal{G}$.

1b. Duality shadow price interpretation: <https://web.stanford.edu/~ashishg/msande111/notes/chapter4.pdf>

2. Slaters Condition Proof Supplementary Material (Proof from BV Book)

$$\begin{aligned}\mu t &\leq \alpha \\ \implies t &\leq \frac{\alpha}{\mu}\end{aligned}$$

If this holds for $t < p^*$, this also holds for $t = p^*$ by the inequality rule for sequences:

$$\lim_{n \rightarrow \infty} a_n = a, \quad \lim_{n \rightarrow \infty} b_n = b, \quad a_n \leq b_n \text{ for all } n \in \mathbb{N} \implies a \leq b.$$

We can set $a_n = p^* - \frac{1}{n}$ and $b_n = \frac{\alpha}{\mu}$. Since $a_n < p^*$

$$p^* \leq \frac{\alpha}{\mu}, \forall n \in \mathbb{N}$$

Also note that for a point $x \in \mathbf{int}\mathcal{D}$, we can travel in any direction within an epsilon length of x , including the direction of a vector from the nullspace of A if it exists.

3. Lagrange Multipliers

Given an optimization problem of the form

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g(x) = 0 \end{aligned}$$

we can use Lagrange multipliers to find the minimum value by solving the following system of equations and plugging in the solution(s):

$$\begin{aligned} \nabla f(x) &= \lambda \nabla g(x) \\ g(x) &= 0 \end{aligned}$$

If $f(x)$ is convex and $g(x)$ is affine, Slater's condition tells us that the duality gap is 0 and we can solve the dual to get the solution:

$$\max_{\nu} \min_x f(x) + \nu g(x)$$

Since $f(x) + \nu g(x)$ is convex, the minimizer x^* (a function of ν) must satisfy

$$\nabla f(x^*) + \nu \nabla g(x^*) = 0$$

Plugging in x^* , we get

$$\begin{aligned} \max_{\nu} \min_x f(x) + \nu g(x) \\ \max_{\nu} f(x^*) + \nu g(x^*) \end{aligned}$$

Note that for any unconstrained optimization problem (even if it is non-convex), the gradient must equal 0:

$$\begin{aligned} 0 &= \nabla_{\nu} f(x^*) + \nabla_{\nu} (\nu g(x^*)) \\ &= \nabla_x f(x^*)^T \nabla_{\nu} x^* + \nu \nabla_{\nu} g(x^*) + g(x^*) \\ &= \nabla_x f(x^*)^T \nabla_{\nu} x^* + \nu \nabla_x g(x^*)^T \nabla_{\nu} x^* + g(x^*) \\ &= (\nabla_x f(x^*) + \nu \nabla_x g(x^*))^T \nabla_{\nu} x^* + g(x^*) \\ &= g(x^*) \end{aligned}$$

Note that

$$\begin{aligned} \nabla f(x^*) + \nu \nabla g(x^*) &= 0 \\ g(x^*) &= 0 \end{aligned}$$

are exactly the equations for the Lagrange multipliers.

Similarly, the KKT conditions (See BV book for proof) state that given an arbitrary optimization problem (does not need to be convex) where $f_0(x), \dots, f_m(x), h_1(x), \dots, h_p(x)$ are differentiable

$$\begin{aligned} \min_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

If the duality gap is 0, x^* is a primal optimal point, and (λ^*, ν^*) is a dual optimal point, then the following must hold:

$$\begin{aligned}
f_i(x^*) &\leq 0, & i &= 1, \dots, m \\
h_i(x^*) &= 0, & i &= 1, \dots, p \\
\lambda_i^* &\geq 0, & i &= 1, \dots, m \\
\lambda_i^* f_i(x^*) &= 0, & i &= 1, \dots, m \quad (\text{complementary slackness})
\end{aligned}$$

$$\nabla f_0(x^*) + \sum_{i=1}^m \lambda_i^* \nabla f_i(x^*) + \sum_{i=1}^p \nu_i^* \nabla h_i(x^*) = 0$$

These are known as the KKT necessary conditions. Note that complementary slackness holds for all optimizations problems where the duality gap is 0.

Note that the KKT conditions are both necessary and sufficient if the problem is convex (e.g. $f_0(x), \dots, f_m(x)$ are convex and $h_1(x), \dots, h_p(x)$ are affine). In other words, if the conditions above hold for any point (x^*, λ^*, ν^*) , then x^* is a solution to the primal problem, (λ^*, ν^*) is a solution to the dual problem, and the duality gap is 0 when the problem is a convex problem.

Note that solving the KKT conditions is the same as solving the equations for the Lagrange multipliers.

2 Mirror Descent

$$y = \underset{y}{\operatorname{argmin}} f(x) + \eta(y - x)^T \nabla f(x) + D_h(y||x)$$

$$D_h(y||x) = h(y) - (h(x) + (y - x)^T \nabla h(x))$$

where h must be strictly convex (this makes sure $D(y||x) = 0 \iff x = y$)

$$y = \underset{y}{\operatorname{argmin}} f(x) + \eta(y - x)^T \nabla f(x) + D_h(y||x)$$

$$\implies \eta \nabla f(x) + \nabla h(y) - \nabla h(x) = 0$$

$$\implies y = \nabla h^{-1}(\nabla h(x) - \eta \nabla f(x))$$

Common Bregman Divergences:

1. $h(x) = cx^T Ax$ where $A \succ 0$

$$\nabla h(x) = 2cAx$$

$$\nabla h^{-1}(x) = \frac{1}{2c} A^{-1} x$$

$$\begin{aligned} D(y||x) &= cy^T Ay - cx^T Ax - (y - x)^T (2cAx) \\ &= cy^T Ay - cx^T Ax - 2cy^T Ax + 2cx^T Ax \\ &= cy^T Ay + cx^T Ax - 2cy^T Ax \\ &= c(y - x)^T A(y - x) \end{aligned}$$

Update rule:

$$\begin{aligned} y &= \nabla h^{-1}(\nabla h(x) - \eta \nabla f(x)) \\ &= \frac{1}{2c} A^{-1} (2cAx - \eta \nabla f(x)) \\ &= x - \frac{\eta}{2c} A^{-1} \nabla f(x) \end{aligned}$$

2. $h(x) = \sum_i x_i \ln(x_i)$ (negative entropy)

$$\nabla h(x)_i = x_i * 1/x_i + \ln(x_i) * 1 = 1 + \ln(x_i)$$

$$\nabla h(x) = \vec{\mathbf{1}} + \ln(x)$$

$$\nabla h^{-1}(x)_i = e^{x_i - 1}$$

$$\nabla h^{-1}(x) = e^{x - \vec{\mathbf{1}}}$$

$$\begin{aligned} D(y||x) &= \sum_i y_i \ln(y_i) - \sum_i x_i \ln(x_i) - (y - x)^T (\vec{\mathbf{1}} + \ln(x)) \\ &= \sum_i y_i \ln(y_i) - \sum_i x_i \ln(x_i) - \sum_i (y_i - x_i) - \sum_i (y_i - x_i) \ln(x_i) \\ &= \sum_i y_i \ln(y_i) - \sum_i x_i \ln(x_i) - \sum_i y_i \ln(x_i) + \sum_i x_i \ln(x_i) + \sum_i (x_i - y_i) \\ &= \sum_i y_i \ln\left(\frac{y_i}{x_i}\right) + \sum_i (x_i - y_i) \\ &= D_{KL}(y||x) + \sum_i (x_i - y_i) \end{aligned}$$

Update rule:

$$\begin{aligned} y &= \nabla h^{-1}(\nabla h(x) - \eta \nabla f(x)) \\ &= \nabla h^{-1}(\vec{\mathbf{1}} + \ln(x) - \eta \nabla f(x)) \\ &= e^{(\vec{\mathbf{1}} + \ln(x) - \eta \nabla f(x)) - \vec{\mathbf{1}}} \\ &= e^{\ln(x) - \eta \nabla f(x)} \\ &= x e^{-\eta \nabla f(x)} \end{aligned}$$

3 Steepest Descent, Conjugate Directions, and Conjugate Gradients

When trying to solve the following a quadratic of the following form

$$\operatorname{argmin}_x f(x) = \frac{1}{2}x^T Ax - b^T x + c$$

where A is symmetric and positive definite, the solution can be found by setting the gradient to 0 and solving for x :

$$\vec{0} = \frac{1}{2}(A + A^T)x - b = Ax - b \implies Ax = b$$

In cases where A has large dimensionality, but is factorizable into a product of smaller matrices, it can be more efficient to use an iterative method like steepest descent or conjugate gradients to solve for x . The method starts at a point x_0 and iteratively updates the point with the point in the at the start of the i th step expressed as x_i . For each point x_i , we can associate an error

$$e_i = x_i - x^*$$

and residual

$$r_i = -\nabla_x f(x_i) = b - Ax = Ae_i$$

The procedure is as follows

1. Initialize x_0
2. $r_0 = b - Ax_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. Determine a search direction d_i
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T A d_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i$
 - 1d. $r_{i+1} = b - Ax_{i+1}$

Note that step 1b and 1c essentially just moves along the search direction d_i starting at x_i to the point $x_{i+1} = x_i + \alpha_i d_i$ that minimizes $f(x_i + \alpha d_i)$. Since f is convex in α ($f(x_i + \alpha d_i)$ is just a single dimension quadratic function in terms of α that has a positive coefficient for α^2 since A is positive definite (e.g.

$\frac{1}{2}d_i^T d_i > 0$), we can solve for α by setting the derivative equal to 0:

$$\begin{aligned}
\frac{d}{d\alpha} f(x_i + \alpha d_i) &= \nabla_x f(x_i + \alpha d_i)^T(d_i) = \vec{0} \quad (\text{The gradient at the next point is orthogonal to } d_i) \\
\implies -r_{i+1}^T d_i &= \vec{0} \quad (\text{this step is just to provide more insight and is not necessary}) \\
\implies (Ax_{i+1} - b)^T d_i &= \vec{0} \quad (\text{this step is just to provide more insight and is not necessary}) \\
\implies (A(x_i + \alpha d_i) - b)^T d_i &= \vec{0} \\
\implies (Ax_i - b + \alpha Ad_i)^T d_i &= \vec{0} \\
\implies (-r_i + \alpha Ad_i)^T d_i &= \vec{0} \\
\implies \alpha d_i^T Ad_i &= r_i^T d_i \\
\implies \alpha &= \frac{r_i^T d_i}{d_i^T Ad_i}
\end{aligned}$$

Note that the chain rule in the first step can be thought of as the Jacobian of $f(x_i + \alpha d_i)^T(d_i)$ with respect to $x_i + \alpha d_i$ times the Jacobian of $x_i + \alpha d_i$ with respect to α . Note that the gradient is transpose of the Jacobian.

Notice if we subtract x^* from both sides of 1a, and pre-multiply both sides by A, we get

$$\begin{aligned}
x_{i+1} - x^* &= x_i - x^* + \frac{d_i^T r_i}{d_i^T Ad_i} d_i \\
\implies e_{i+1} &= e_i + \frac{d_i^T r_i}{d_i^T Ad_i} d_i \implies Ae_{i+1} = Ae_i + \frac{d_i^T r_i}{d_i^T Ad_i} Ad_i \\
&\implies -r_{i+1} = -r_i + \frac{d_i^T r_i}{d_i^T Ad_i} Ad_i \\
&\implies r_{i+1} = r_i - \frac{d_i^T r_i}{d_i^T Ad_i} Ad_i
\end{aligned}$$

meaning we can simplify the algorithm the following

1. Initialize x_0
2. $r_0 = b - Ax_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. Determine a search direction d_i
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T Ad_i} \implies \alpha_i = -\frac{d_i^T Ae_i}{d_i^T Ad_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i \implies e_{i+1} = e_i + \alpha_i d_i$
 - 1d. $r_{i+1} = r_i - \alpha_i Ad_i$

Notice how modified step 1d, which now relies on the product Ad_i rather than Ax_i . Since 1b also uses Ad_i , which can be cached, we now only need to rely

on one matrix multiplication. Note that due to the floating point roundoff, we usually want to compute r_{i+1} using the old step 1d.

Any solution to $Ax = b$ satisfies $A^T Ax = A^T b$, so we can also apply the algorithm to solve $A^T Ax = A^T b$ when A is not symmetric.

3.1 Steepest Descent

From above:

1. Initialize x_0
2. $r_0 = b - Ax_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. Determine a search direction d_i
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T A d_i} \implies \alpha_i = -\frac{d_i^T A e_i}{d_i^T A d_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i \implies e_{i+1} = e_i + \alpha_i d_i$
 - 1d. $r_{i+1} = r_i - \alpha_i A d_i$

Steepest descent algorithm just sets d_i equal to r_i . Note that the algorithm also works to find a least squares solution to $\min_x \|Ax - b\|^2$. Note that whenever $d_i = r_i = b - Ax_i$ is in the nullspace of A , this means that $A^T(Ax - b) = 0$, meaning x_i is a solution to the least squares problem. Note that we can keep moving in the direction of r_i to decrease $\frac{1}{2}x^T Ax - b^T x + c$, but this will not decrease $\|Ax - b\|^2$ further. This is also not guaranteed to produce the min norm solution. For example, stopping when $Ar_i = \vec{0}$ for the following set up

$$x_0 = \vec{0}$$

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

gives

$$x^* = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

whereas the min norm solution solution to least squares is

$$x^* = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

3.2 Conjugate Descent Methods

3.2.1 Conjugate Gram Schmidt

v_1 and v_2 being A orthogonal means that

$$v_1^T A v_2 = \vec{0}$$

$$\implies v_1^T (V \Sigma^{\frac{1}{2}} \Sigma^{\frac{1}{2}T} V^T) v_2 = (\Sigma^{\frac{1}{2}} V^T v_1)^T (\Sigma^{\frac{1}{2}} V^T) v_2 \vec{0}$$

$$\implies v_1^T (\Sigma^{\frac{1}{2}} V^T \Sigma^{\frac{1}{2}} V^T) v_2 = (V^T \Sigma^{\frac{1}{2}} V^T v_1)^T (V^T \Sigma^{\frac{1}{2}} V^T v_2) \vec{0}$$

where the last two lines show that v_1 and v_2 being A orthogonal means they are orthogonal in a transformed space under the Cholesky decomposition and square root of A .

Given a basis $R = \{r_0, \dots, r_m\}$ for any m dimensional subspace, where each r_i has dimension $n > m$, we can construct a basis of m A orthogonal vectors $D = \{d_0, \dots, d_m\}$ such that $\text{Span}\{r_0, \dots, r_m\} = \text{Span}\{d_0, \dots, d_m\}$ using conjugate Gram-Schmidt. We can detail the method and prove it is correct by induction as follows:

Base case: $m = 1$

$$D = r_0$$

Induction hypothesis: We can construct a basis of m A orthogonal vectors $D = \{d_0, \dots, d_m\}$ such that $\text{Span}\{r_0, \dots, r_m\} = \text{Span}\{d_0, \dots, d_m\}$ for $m = k$.

Inductive Step: Prove for $m = k + 1$.

Given a set of $m = k + 1$ vectors $R_{k+1} = \{r_0, \dots, r_{k+1}\}$, we know that we can construct a basis of $m = k$ A orthogonal vectors $D_k = \{d_0, \dots, d_k\}$ such that $\text{Span}(R_k) = \text{Span}(\{r_0, \dots, r_k\}) = \text{Span}(D_k)$. Note that

$$\text{Span}(R_k \cup \{r_{k+1}\}) = \text{Span}(D_k \cup \{r_{k+1}\}) = \text{Span}(\{d_0, \dots, d_k, r_{k+1}\})$$

The $\text{Span}(\{d_0, \dots, d_k, r_{k+1}\})$ also remains the same if we replace r_{k+1} with a vector d_{k+1} that is the sum of r_{k+1} and a linear combination of other vectors in the basis

$$d_{k+1} = r_{k+1} + \sum_{i=0}^k c_i d_i$$

If we set

$$c_i = \begin{cases} -\beta_i = -\frac{d_i^T A r_{k+1}}{d_i^T A d_i}, & \text{if } d_i^T A d_i > 0 \\ 0, & \text{if } d_i^T A d_i = 0 \end{cases}$$

we get that for any $j \leq k$

$$\begin{aligned} d_j^T A d_{k+1} &= d_j^T A r_{k+1} + \sum_{i=0}^k -\beta d_j^T A d_i \\ &= d_j^T A r_{k+1} - \beta_j d_j^T A d_j \quad (\text{since the previous } k \text{ vectors were mutually } A \text{ orthogonal}) \end{aligned}$$

For the case where $d_j^T A d_j > 0$:

$$\begin{aligned} d_j^T A d_{k+1} &= d_j^T A r_{k+1} - \beta_j d_j^T A d_j \\ &= d_j^T A r_{k+1} - \frac{d_j^T A r_{k+1}}{d_j^T A d_j} d_j^T A d_j \\ &= d_j^T A r_{k+1} - d_j^T A r_{k+1} \\ &= 0 \end{aligned}$$

And in the case where

$$d_j^T A d_j = 0 \implies \|A^{\frac{1}{2}} d_j\|^2 = 0 \implies A^{\frac{1}{2}} d_j = 0 \implies A^{\frac{1}{2}}(A^{\frac{1}{2}} d_j) = A d_j = 0$$

$$\begin{aligned}
d_j^T Ad_{k+1} &= d_j^T Ar_{k+1} - \beta_j d_j^T Ad_j \\
&= (A^T d_j)^T r_{k+1} - 0 \\
&= (Ad_j)^T r_{k+1} \\
&= 0
\end{aligned}$$

Thus, we get that $D_{k+1} = \{d_0, \dots, d_k, d_{k+1}\}$ is a set of mutually A orthogonal vectors such that $\text{Span}(R_{k+1}) = \text{Span}(D_{k+1})$ as desired. As a side note, we can form an orthonormal basis from the eigenvectors of A by the spectral theorem and this eigenvector basis is also mutually A orthogonal.

3.2.2 Conjugate Directions

From above:

1. Initialize x_0
2. $r_0 = b - Ax_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. Determine a search direction d_i
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T Ad_i} \implies \alpha_i = -\frac{d_i^T Ae_i}{d_i^T Ad_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i \implies e_{i+1} = e_i + \alpha_i d_i$
 - 1d. $r_{i+1} = r_i - \alpha_i Ad_i$

The conjugate directions algorithm sets d_i to a set of n (where n is the dimensionality of x) A orthogonal vectors that make up a basis. Conjugate directions converges in at most n steps, which can be shown as follows.

Since the d_i form a basis in conjugate directions, any arbitrary vector v can be constructed as a linear combination

$$v = \sum_{i=0}^{n-1} \beta_i d_i$$

and to get the coefficient β_j , we can premultiply both sides by $d_j^T A$:

$$\begin{aligned}
d_j^T Av &= \sum_{i=0}^{n-1} \beta_j d_j^T Ad_i = \beta_j d_j^T Ad_j \\
\implies \beta_j &= \frac{d_j^T Av}{d_j^T Ad_j}
\end{aligned}$$

Since the d_i form a basis, we know that the initial error can be expressed as

$$e_0 = \sum_{i=0}^{n-1} \beta_i d_i$$

and we see that in step 1c, $\alpha_i = -\beta_i$ when $d_i^T A d_i > 0 \implies A^{\frac{1}{2}} d_i \neq 0 \implies A d_i \neq 0$. This means that

$$\begin{aligned}
e_i &= e_0 + \sum_{j=0}^{i-1} \alpha_j d_j \\
&= \sum_{j=0}^{n-1} \beta_j d_j + \sum_{j=0}^{i-1} \alpha_j d_j \\
&= \sum_{j=0}^{n-1} \beta_j d_j - \sum_{j=0}^{i-1} \beta_j \mathbf{1}\{d_j \notin \text{Null}(A)\} \beta_j \mathbf{1}\{d_j \notin \text{Null}(A)\} d_j \\
&= \sum_{j=0}^{i-1} \mathbf{1}\{d_j \in \text{Null}(A)\} \beta_j d_j + \sum_{j=i}^{n-1} \beta_j d_j
\end{aligned}$$

For $\sum_{j=0}^{i-1} \mathbf{1}\{d_j \in \text{Null}(A)\} \beta_j d_j$, which equals $\vec{0}$ when A is positive definite as desired. When A is positive semidefinite, e_n is linear combination of vectors in $\text{Null}(A)$, meaning

$$\begin{aligned}
A e_n &= \vec{0} \\
\implies A(x_n - x^*) &= A x_n - b = \vec{0} \\
\implies A^T(A x_n - b) &= \vec{0}
\end{aligned}$$

meaning e_n is a least squares solution.

Notice that the error e_i is always A orthogonal to the previous search directions $d_0, \dots, d_{i-1}$. In addition to using the expression above, we can also prove this by induction where $d_0^T A e_1 = r_0^T r_1 = 0$. In iteration $k+1$,

$$\begin{aligned}
d_k^T r_{k+1} &= 0 \quad (\text{search direction is orthogonal to next residual (e.g. negative gradient) for optimal } \alpha_k) \\
\implies d_k^T A e_{k+1} &= 0
\end{aligned}$$

For $j < k$

$$\begin{aligned}
d_j^T A e_{k+1} &= d_j^T A(e_k + \alpha_k d_k) \\
&= d_j^T A e_k + d_j^T A d_k \\
&= d_j^T A e_k \\
&= 0
\end{aligned}$$

where the last step comes from using the inductive hypothesis. Similarly, this tells us that e_n is A orthogonal to all old search directions, which formed a basis; thus, e_n is A orthogonal to all vectors, meaning $e_n = \vec{0}$ or $e_n \in \text{Null}(A)$

e_i being A orthogonal to old search directions also means that the residual $r_i = -A e_i$ is orthogonal to the previous search directions $d_0, \dots, d_{i-1}$.

3.3 Conjugate Gradient

The conjugate gradient algorithm is just a version of conjugate directions, where the mutually A orthogonal search directions are built from running the conjugate Gram Schmidt procedure on residuals (e.g. the negative gradients) encountered in each iteration of the algorithm. From before

1. Initialize x_0
2. $r_0 = b - Ax_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. Determine a search direction d_i
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T A d_i} \implies \alpha_i = -\frac{d_i^T A e_i}{d_i^T A d_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i \implies e_{i+1} = e_i + \alpha_i d_i$
 - 1d. $r_{i+1} = r_i - \alpha_i A d_i$

now becomes

1. Initialize x_0
2. $r_0 = b - Ax_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. $d_i = r_i + \sum_{j=0}^{i-1} c_j d_j$
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T A d_i} \implies \alpha_i = -\frac{d_i^T A e_i}{d_i^T A d_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i \implies e_{i+1} = e_i + \alpha_i d_i$
 - 1d. $r_{i+1} = r_i - \alpha_i A d_i$

where

$$c_j = \begin{cases} -\beta_j = -\frac{d_j^T A r_j}{d_j^T A d_j}, & \text{if } d_j^T A d_j > 0 \\ 0, & \text{if } d_j^T A d_j = 0 \end{cases}$$

In each iteration i , we know that the residual r_i is orthogonal and independent to all old search directions (property of the conjugate directions as shown above). Let

$$\begin{aligned} R_i &= \{r_0, \dots, r_{i-1}\} \\ D_i &= \{d_0, \dots, d_{i-1}\} \end{aligned}$$

By the proof of conjugate Gram Schmidt from above

$$\text{Span}(R_i) = \text{Span}(D_i)$$

In step 1d, we have $r_i = r_{i-1} - \alpha_{i-1}Ad_{i-1}$. meaning

$$\begin{aligned} \text{Span}(D_i) &= \text{Span}(R_i) \\ &= \text{Span}(\{r_0, \dots, r_{i-1}, Ad_{i-1}\}) \\ &= \text{Span}(\{d_0, \dots, d_{i-1}, Ad_{i-1}\}) \end{aligned}$$

Applying this iteratively starting from $i = 0$ or using induction, we get that

$$\begin{aligned} \text{Span}(D_i) &= \text{Span}(\{d_0, Ad_0, \dots, A^{i-1}d_0\}) = \text{Span}(\{d_0, Ad_0, Ad_1 \dots, Ad_{i-2}\}) \\ \iff \text{Span}(R_i) &= \text{Span}(\{r_0, Ar_0, \dots, A^{i-1}r_0\}) = \text{Span}(\{r_0, Ar_0, Ar_1 \dots, Ar_{i-2}\}) \end{aligned}$$

This because each r_i can be constructed by a linear combination of d_j for $j \leq i$, Ar_i can be constructed by the same linear combination of Ad_j , meaning we can replace each d_i in the basis for the span with r_i while keeping the span the same. The subspaces $\text{Span}(D_i)$ and $\text{Span}(R_i)$ are called Krylov subspaces, which are constructed from the span of a set of vectors comprised of a single vector that is repeatedly multiplied by a matrix.

In general conjugate directions, we showed that r_{i+1} is orthogonal to all previous search directions in D_{i+1} , meaning r_{i+1} is orthogonal to $\text{Span}(D_i) = \text{Span}(\{d_0, Ad_0, \dots, Ad_{i-2}\}) (= \text{Span}(R_i))$. This also tells us that r_i is A orthogonal to $\text{Span}(D_{i-1}) = \text{Span}(\{d_0, \dots, d_{i-2}\})$. As a result, we only need to subtract out the the component from d_{i-1} from r_i to make it A orthogonal to all previous search directions $d_0, \dots, d_{i-1}$ (e.g. last step of conjugate Gram Schmidt).

Thus, we can change step 1a from

$$d_i = r_i + \sum_{i=0}^{i-1} c_i d_i$$

to

$$d_i = r_i + c_{i-1}d_{i-1}$$

Remember that

$$c_{i-1} = \begin{cases} -\beta_{i-1} = -\frac{d_{i-1}^T Ar_i}{d_{i-1}^T Ad_{i-1}}, & \text{if } d_{i-1}^T Ad_{i-1} > 0 \\ 0, & \text{if } d_{i-1}^T Ad_{i-1} = 0 \end{cases}$$

Since r_i is orthogonal to all previous residuals and search directions, we can simplify the calculation for $-\beta_{i-1}$ as follows

$$\begin{aligned}
-\beta_{i-1} &= -\frac{d_{i-1}^T A r_i}{d_{i-1}^T A d_{i-1}} \\
&= -\frac{1}{\alpha_{i-1}} \frac{(\alpha_{i-1} A d_{i-1})^T r_i + 0}{d_{i-1}^T A d_{i-1}} \\
&= -\frac{1}{\alpha_{i-1}} \frac{(\alpha_{i-1} A d_{i-1})^T r_i + r_{i-1}^T r_i}{d_{i-1}^T A d_{i-1}} \\
&= -\frac{1}{\alpha_{i-1}} \frac{(r_{i-1} + \alpha_{i-1} A d_{i-1})^T r_i}{d_{i-1}^T A d_{i-1}} \\
&= -\frac{1}{\alpha_{i-1}} \frac{r_i^T r_i}{d_{i-1}^T A d_{i-1}} \\
&= \frac{1}{\frac{d_{i-1}^T r_{i-1}}{d_{i-1}^T A d_{i-1}}} \frac{r_i^T r_i}{d_{i-1}^T A d_{i-1}} \text{ (from step 1b)} \\
&= \frac{d_{i-1}^T A d_{i-1}}{d_{i-1}^T r_{i-1}} \frac{r_i^T r_i}{d_{i-1}^T A d_{i-1}} \\
&= \frac{r_i^T r_i}{d_{i-1}^T r_{i-1}} \\
&= \frac{r_i^T r_i}{d_{i-1}^T r_{i-1} + 0} \\
&= \frac{r_i^T r_i}{d_{i-1}^T r_{i-1} - c_{i-2} d_{i-2}^T r_{i-1}} \text{ (residual is orthogonal to old search directions)} \\
&= \frac{r_i^T r_i}{(d_{i-1} - c_{i-2} d_{i-2})^T r_{i-1}} \text{ (from initial change to step 1a above)} \\
&= \frac{r_i^T r_i}{r_{i-1}^T r_{i-1}}
\end{aligned}$$

The final algorithm is as follows

1. Initialize x_0
2. $r_0 = b - A x_0$
3. Repeat until convergence for $i = 0, \dots$
 - 1a. $d_i = r_i + c_{i-1} d_{i-1}$
 - 1b. $\alpha_i = \frac{d_i^T r_i}{d_i^T A d_i} \implies \alpha_i = -\frac{d_i^T A e_i}{d_i^T A d_i}$
 - 1c. $x_{i+1} = x_i + \alpha_i d_i \implies e_{i+1} = e_i + \alpha_i d_i$
 - 1d. $r_{i+1} = r_i - \alpha_i A d_i$

where

$$c_{i-1} = \begin{cases} -\beta_i = -\frac{d_{i-1}^T Ar_{i-1}}{d_{i-1}^T Ad_{i-1}}, & \text{if } d_{i-1}^T Ad_{i-1} > 0 \\ 0, & \text{if } d_{i-1}^T Ad_{i-1} = 0 \end{cases}$$

Notice how we only ever need to compute Ad_i as the only operation with A (the result Ad_i in step 1d can also be saved for the next iteration for computing c_i in step 1a), meaning the cost is dominated by n matrix multiplications with a vector of length n for a cost of $O(n^3)$. Note if the matrix is broken up into $A = JMJ$ where J is $n \times m$ and M is $m \times m$, the time complexity is $O(mn^2 + m^2n)$.

For positive semi-definite matrices, if $d_i^T Ad_i = 0 \implies Ad_i = 0$ (which was formed from adding the r_i and a linear combination of old search directions, it does not mean that $Ar_i = 0$ (in which case we have already found the least squares solution and can stop early like in steepest descent). We still have to run at least n iterations like in conjugate directions to get the least squares solution.

4 Minimizing the Linear Approximation Given Constraints

4.1 Distance Constraint

Besides adding the penalty, we can use it as a constraint in an optimization problem

$$\begin{aligned} y &= \underset{y}{\operatorname{argmin}} \quad f(x) + (y - x)^T \nabla f(x) \\ \text{subject to} \quad & D_h(y||x) \leq \epsilon \\ &= \underset{y}{\operatorname{argmin}} \quad (y - x)^T \nabla f(x) \\ \text{subject to} \quad & D_h(y||x) \leq \epsilon \end{aligned}$$

One example is $h(x) = cx^T Ax, A \succ 0$

$$\begin{aligned} y &= \underset{y}{\operatorname{argmin}} \quad (y - x)^T \nabla f(x) \\ \text{subject to} \quad & c(y - x)^T A(y - x) \leq \epsilon \end{aligned}$$

3.1a. Taking the dual normally:

The problem is convex and the point $x + \sqrt{\frac{\epsilon}{2\lambda_1}} v_1$, where v_1 is a unit length eigenvector of A and λ_1 is its associated eigenvalue, strictly satisfies the inequality constraint. By Slater's condition, strong duality holds. Thus, y is the optimizer of inner min of

$$\max_{\lambda \geq 0} \min_y \quad (y - x)^T \nabla f(x) + \lambda(c(y - x)^T A(y - x) - \epsilon)$$

Since the inner min is convex with respect to y , we can set the gradient with respect to y to 0:

$$\begin{aligned} \nabla f(x) + 2\lambda c A(y - x) &= 0 \\ \implies y - x &= -\frac{A^{-1} \nabla f(x)}{2\lambda c} \\ \implies y &= x - \frac{A^{-1} \nabla f(x)}{2\lambda c} \end{aligned}$$

Plugging this in to the dual, we get

$$\begin{aligned} \max_{\lambda \geq 0} \quad & \left(x - \frac{A^{-1} \nabla f(x)}{2\lambda c} - x \right)^T \nabla f(x) + \lambda \left(c \left(x - \frac{A^{-1} \nabla f(x)}{2\lambda c} - x \right)^T A \left(x - \frac{A^{-1} \nabla f(x)}{2\lambda c} - x \right) - \epsilon \right) \\ &= \max_{\lambda \geq 0} \quad \left(-\frac{A^{-1} \nabla f(x)}{2\lambda c} \right)^T \nabla f(x) + \lambda \left(c \left(-\frac{A^{-1} \nabla f(x)}{2\lambda c} \right)^T A \left(-\frac{A^{-1} \nabla f(x)}{2\lambda c} \right) - \epsilon \right) \\ &= \max_{\lambda \geq 0} \quad -\frac{1}{2\lambda c} \nabla f(x)^T A^{-1} \nabla f(x) + \frac{-1}{2} \nabla f(x)^T A^{-1} A \left(-\frac{A^{-1} \nabla f(x)}{2\lambda c} \right) - \lambda \epsilon \end{aligned}$$

$$\begin{aligned}
&= \max_{\lambda \geq 0} -\frac{1}{2\lambda c} \nabla f(x)^T A^{-1} \nabla f(x) + \frac{1}{4\lambda c} \nabla f(x)^T A^{-1} \nabla f(x) - \lambda \epsilon \\
&= \max_{\lambda \geq 0} -\frac{1}{4\lambda c} \nabla f(x)^T A^{-1} \nabla f(x) - \lambda \epsilon
\end{aligned}$$

Since the expression being maximized is concave for $\lambda \geq 0$, we can set the gradient to 0 to solve for the optimizer:

$$\begin{aligned}
&\frac{1}{4\lambda^2 c} \nabla f(x)^T A^{-1} \nabla f(x) - \epsilon = 0 \\
&\implies 4\lambda^2 c \epsilon = \nabla f(x)^T A^{-1} \nabla f(x) \\
&\implies \lambda = \sqrt{\frac{\nabla f(x)^T A^{-1} \nabla f(x)}{4c\epsilon}} \\
&= \sqrt{\frac{\nabla f(x)^T A^{-1} \nabla f(x)}{4c\epsilon}}
\end{aligned}$$

Plugging this value of λ for y , we get

$$\begin{aligned}
y &= x - \frac{A^{-1} \nabla f(x)}{2\lambda c} \\
&= x - \frac{A^{-1} \nabla f(x)}{2\sqrt{\frac{\nabla f(x)^T A^{-1} \nabla f(x)}{4c\epsilon}} c} \\
&= x - \sqrt{\frac{\epsilon}{c \nabla f(x)^T A^{-1} \nabla f(x)}} A^{-1} \nabla f(x)
\end{aligned}$$

3.1b. Using KKT conditions:

$$\begin{aligned}
y &= \underset{y}{\operatorname{argmin}} (y - x)^T \nabla f(x) \\
&\text{subject to } c(y - x)^T A(y - x) \leq \epsilon
\end{aligned}$$

Since the problem is convex, we just need to find any point that satisfies the KKT conditions will be an optimizer:

$$c(y - x)^T A(y - x) - \epsilon \leq 0 \quad (1)$$

$$\lambda \geq 0 \quad (2)$$

$$\lambda(c(y - x)^T A(y - x) - \epsilon) = 0 \quad (3)$$

$$\nabla((y - x)^T \nabla f(x)) + \lambda \nabla(c(y - x)^T A(y - x) - \epsilon) = 0 \quad (4)$$

Note that the condition (4) is what we solved previously:

$$\nabla((y - x)^T \nabla f(x)) + \lambda \nabla(c(y - x)^T A(y - x) - \epsilon) = \nabla f(x) + 2\lambda c A(y - x) = 0$$

$$\implies y = x - \frac{A^{-1} \nabla f(x)}{2\lambda c}$$

Plugging this into (3), we get that

$$\begin{aligned}
0 &= \lambda(c(y-x)^T A(y-x) - \epsilon) \\
&= \lambda \left(c \left(x - \frac{A^{-1} \nabla f(x)}{2\lambda c} - x \right)^T A \left(x - \frac{A^{-1} \nabla f(x)}{2\lambda c} - x \right) - \epsilon \right) \\
&= \frac{1}{4\lambda c} \nabla f(x)^T A^{-1} \nabla f(x) - \lambda \epsilon \\
\implies 0 &= \frac{1}{4c} \nabla f(x)^T A^{-1} \nabla f(x) - \lambda^2 \epsilon \\
\implies \lambda &= \pm \sqrt{\frac{\nabla f(x)^T A^{-1} \nabla f(x)}{4c\epsilon}}
\end{aligned}$$

Since $\lambda \geq 0$:

$$\lambda = \sqrt{\frac{\nabla f(x)^T A^{-1} \nabla f(x)}{4c\epsilon}}$$

Plugging in λ for y , we get

$$\begin{aligned}
y &= x - \frac{A^{-1} \nabla f(x)}{2\lambda c} \\
&= x - \sqrt{\frac{\epsilon}{c \nabla f(x)^T A^{-1} \nabla f(x)}} A^{-1} \nabla f(x)
\end{aligned}$$

just like before.

4.2 Multiplicative Weights

We can also add a sum constraint to mirror descent using the Bregman divergence with $h(x) = \sum_i x_i \ln(x_i)$ to get multiplicative weights:

$$\begin{aligned}
&\underset{y}{\operatorname{argmin}} && f(x) + \eta(y-x)^T \nabla f(x) + \sum_i y_i \ln\left(\frac{y_i}{x_i}\right) + \sum_i (x_i - y_i) \\
&\text{subject to} && \bar{\mathbf{1}}^T y = 1 \\
&&& y \geq 0
\end{aligned}$$

This is the same as

$$\begin{aligned}
&\underset{y}{\operatorname{argmin}} && f(x) + \eta(y-x)^T \nabla f(x) + \sum_i y_i \ln\left(\frac{y_i}{x_i}\right) + \sum_i (x_i - y_i) \\
&\text{subject to} && \bar{\mathbf{1}}^T y - 1 = 0 \\
&&& -y \leq 0
\end{aligned}$$

Since the problem is convex for $y \geq 0$ (Hessian is $\operatorname{diag}(\frac{1}{y_i})$), any point (if it exists) that satisfies the KKT conditions is a solution with 0 duality gap.

$$-y \leq 0 \quad (1)$$

$$\lambda \geq 0 \quad (2)$$

$$\vec{\mathbf{1}}^T y - 1 = 0 \quad (3)$$

$$\lambda(-y) = \vec{\mathbf{0}} \quad (4)$$

$$\nabla(f(x) + \eta(y - x)^T \nabla f(x) + \sum_i y_i \ln\left(\frac{y_i}{x_i}\right) - \sum_i (x_i - y_i)) + \lambda^T \nabla(-y) + \nu \nabla(\vec{\mathbf{1}}^T y - 1) = 0 \quad (5)$$

Starting with condition (5):

$$\begin{aligned} \nabla \left(f(x) + \eta(y - x)^T \nabla f(x) + \sum_i y_i \ln\left(\frac{y_i}{x_i}\right) + \sum_i (x_i - y_i) \right) - \nabla \lambda^T y + \nu \nabla(\vec{\mathbf{1}}^T y - 1) &= 0 \\ &= \eta \nabla((y - x)^T \nabla f(x)) + \nabla \sum_i y_i \ln\left(\frac{y_i}{x_i}\right) - \nabla \sum_i y_i - \lambda + \nu \vec{\mathbf{1}} \\ &= \eta \nabla f(x) + \vec{\mathbf{1}} + \ln\left(\frac{y}{x}\right) - \vec{\mathbf{1}} - \lambda + \nu \vec{\mathbf{1}} \\ &= \eta \nabla f(x) + \ln\left(\frac{y}{x}\right) - \lambda + \nu \vec{\mathbf{1}} \\ \implies \ln\left(\frac{y}{x}\right) &= -\eta \nabla f(x) + \lambda - \nu \vec{\mathbf{1}} \\ \implies y &= x e^{-\eta \nabla f(x) + \lambda - \nu \vec{\mathbf{1}}} \end{aligned}$$

Plugging this into condition (3), we get:

$$\begin{aligned} \vec{\mathbf{1}}^T y - 1 &= 0 \\ &= \vec{\mathbf{1}}^T (x e^{-\eta \nabla f(x) + \lambda - \nu \vec{\mathbf{1}}}) - 1 = 0 \\ \implies \sum_i x_i e^{-\eta \nabla f(x)_i + \lambda_i} e^{-\nu} &= 1 \\ \implies e^\nu &= \sum_i x_i e^{-\eta \nabla f(x)_i + \lambda_i} \\ \implies \nu &= \ln\left(\sum_i x_i e^{-\eta \nabla f(x)_i + \lambda_i}\right) \end{aligned}$$

Plugging this in for y :

$$\begin{aligned} y &= x e^{-\eta \nabla f(x) + \lambda - \nu \vec{\mathbf{1}}} \\ &= x e^{-\eta \nabla f(x) + \lambda - \ln(\sum_i x_i e^{-\eta \nabla f(x)_i + \lambda_i})} \vec{\mathbf{1}} \\ &= \frac{x e^{-\eta \nabla f(x) + \lambda}}{(\sum_i x_i e^{-\eta \nabla f(x)_i + \lambda_i})} \vec{\mathbf{1}} \end{aligned}$$

$$\begin{aligned}
&= \frac{xe^{-\eta \nabla f(x) + \lambda}}{\sum_i x_i e^{-\eta \nabla f(x)_i + \lambda_i}} \\
&= \frac{xe^{-\eta \nabla f(x) + \lambda}}{(xe^{-\eta \nabla f(x) + \lambda})^T \vec{\mathbf{1}}}
\end{aligned}$$

To solve for λ , we can use condition (4).

Case 1: $x_i \neq 0$

In this case, $y_i \neq 0$ assuming the gradient is bounded, so $\lambda_i = 0$ and

$$y_i = \frac{x_i e^{-\eta \nabla f(x)_i}}{\sum_i x_i e^{-\eta \nabla f(x)_i}}$$

Case 2: $x_i = 0$

In this case, λ can be anything. However, $y_i = 0$ regardless of what λ_i is, so we still have

$$y_i = \frac{x_i e^{-\eta \nabla f(x)_i}}{\sum_i x_i e^{-\eta \nabla f(x)_i}}$$

In both cases, all valid λ that satisfy the KKT conditions do not impact y (and also ν by similar reasoning). Thus, for all points (y and ν will always be the same but λ may vary) that satisfy the KKT conditions:

$$y = \frac{xe^{-\eta \nabla f(x)}}{(xe^{-\eta \nabla f(x)})^T \vec{\mathbf{1}}}$$

5 Natural Gradient

In RL, a typical update rule to maximize the linear approximation of the expected return while limiting how big of a step we can take is defined as

$$\begin{aligned} \theta' &\leftarrow \arg \max_{\theta'} (\theta' - \theta)^T \nabla_{\theta} J(\theta) \\ \text{subject to } &D_{KL}(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s)) \leq \epsilon \quad \forall s \end{aligned}$$

Notice how the new parameters are set to the modeled parameters in the KL divergence constraint unlike with mirror descent using KL divergence as the Bregman divergence.

Here, we can approximate $D_{KL}(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s))$ using its second order approximation:

$$\begin{aligned} D(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s)) &= \int_a \pi_{\theta}(a|s) \log \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta'}(a|s)} \right) \\ \nabla_{\theta'} D_{KL}(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s)) &= \nabla_{\theta'} \int_a \pi_{\theta}(a|s) \log \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta'}(a|s)} \right) \\ &= \int_a \pi_{\theta}(a|s) \nabla_{\theta'} \log \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta'}(a|s)} \right) \\ &= \int_a \pi_{\theta}(a|s) \frac{\pi_{\theta'}(a|s) - \pi_{\theta}(a|s)}{\pi_{\theta}(a|s) p_{\pi_{\theta'}(x)}^2} \nabla_{\theta'} \pi_{\theta'}(a|s) \\ &= \int_a \frac{-\pi_{\theta}(a|s)}{\pi_{\theta'}(a|s)} \nabla_{\theta'} \pi_{\theta'}(a|s) \\ &= \int_a -\pi_{\theta}(a|s) \nabla_{\theta'} \log(\pi_{\theta'}(a|s)) \\ &= -\mathbb{E}_{a \sim \pi_{\theta}(a|s)} [\nabla_{\theta'} \log(\pi_{\theta'}(a|s))] \\ \nabla_{\theta'}^2 D_{KL}(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s)) &= \nabla_{\theta'} \int_a -\pi_{\theta}(a|s) \nabla_{\theta'} \log(\pi_{\theta'}(a|s)) \\ &= \int_a -\pi_{\theta}(a|s) \nabla_{\theta'}^2 \log(\pi_{\theta'}(a|s)) \\ &= -\mathbb{E}_{a \sim \pi_{\theta}(a|s)} [\nabla_{\theta'}^2 \log(\pi_{\theta'}(a|s))] \end{aligned}$$

Note that

$$\begin{aligned} -\mathbb{E}_{a \sim \pi_{\theta}(a|s)} [\nabla_{\theta'}^2 \log(\pi_{\theta'}(a|s))] &= \mathbb{E}_{a \sim \pi_{\theta}(a|s)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a|s)) \nabla_{\theta'} \log(\pi_{\theta'}(a|s))^T \right] \\ \iff -\mathbb{E}_{a \sim \pi_{\theta}(a|s)} [\nabla_{\theta'}^2 \log(\pi_{\theta'}(a|s))]_{ij} &= \mathbb{E}_{a \sim \pi_{\theta}(a|s)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a|s)) \nabla_{\theta'} \log(\pi_{\theta'}(a|s))^T \right]_{ij} \\ \iff -\mathbb{E}_{a \sim \pi_{\theta}(a|s)} \left[\frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] &= \mathbb{E}_{a \sim \pi_{\theta}(a|s)} \left[\frac{\partial}{\partial \theta'_i} \log(\pi_{\theta'}(a|s)) \frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] \end{aligned}$$

Starting with the expression on the left

$$\begin{aligned}
& -\mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] = -\mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \left(\frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right) \right] \\
& = -\mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \left(\frac{1}{\pi_{\theta'}(a|s)} * \frac{\partial}{\partial \theta'_j} \pi_{\theta'}(a|s) \right) \right] \\
& = -\mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{1}{\pi_{\theta'}(a|s)} \frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \pi_{\theta'}(a|s) + \frac{-1}{\pi_{\theta'}(a|s)^2} \frac{\partial}{\partial \theta'_i} \pi_{\theta'}(a|s) \frac{\partial}{\partial \theta'_j} \pi_{\theta'}(a|s) \right] \\
& = -\mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{1}{\pi_{\theta'}(a|s)} \frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \pi_{\theta'}(a|s) \right] + \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{1}{\pi_{\theta'}(a|s)^2} \frac{\partial}{\partial \theta'_i} \pi_{\theta'}(a|s) \frac{\partial}{\partial \theta'_j} \pi_{\theta'}(a|s) \right] \\
& = -\int_a \pi_\theta(a|s) \frac{1}{\pi_{\theta'}(a|s)} \frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \pi_{\theta'}(a|s) + \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \log(\pi_{\theta'}(a|s)) \frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] \\
& = -\int_a \frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \pi_{\theta'}(a|s) + \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \log(\pi_{\theta'}(a|s)) \frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] \\
& = -\frac{\partial^2}{\partial \theta'_i \partial \theta'_j} \int_a \pi_{\theta'}(a|s) + \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \log(\pi_{\theta'}(a|s)) \frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] \\
& = -\frac{\partial^2}{\partial \theta'_i \partial \theta'_j} 1 + \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \log(\pi_{\theta'}(a|s)) \frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] \\
& = \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\frac{\partial}{\partial \theta'_i} \log(\pi_{\theta'}(a|s)) \frac{\partial}{\partial \theta'_j} \log(\pi_{\theta'}(a|s)) \right] \\
& \iff -\mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_{\theta'}^2 \log(\pi_{\theta'}(a|s))] = \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_{\theta'} \log(\pi_{\theta'}(a|s)) \nabla_{\theta'} \log(\pi_{\theta'}(a|s))^T]
\end{aligned}$$

We can then approximate the KL divergence around $\theta' = \theta$ as

$$\begin{aligned}
D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s)) & \approx (\theta' - \theta)^T \nabla_{\theta'} D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s))|_{\theta'=\theta} \\
& + \frac{1}{2} (\theta' - \theta)^T \nabla_{\theta'}^2 D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s))|_{\theta'=\theta} (\theta' - \theta) \\
& = (\theta' - \theta)^T (-\mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_{\theta'} \log(\pi_{\theta'}(a|s))]) \\
& + \frac{1}{2} (\theta' - \theta)^T \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_{\theta'} \log(\pi_{\theta'}(a|s))|_{\theta'=\theta} \nabla_{\theta'} \log(\pi_{\theta'}(a|s))^T|_{\theta'=\theta}] (\theta' - \theta) \\
& = (\theta' - \theta)^T \vec{\mathbf{0}} + \frac{1}{2} (\theta' - \theta)^T \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_{\theta'} \log(\pi_{\theta'}(a|s)) \nabla_{\theta'} \log(\pi_{\theta'}(a|s))^T] (\theta' - \theta) \\
& = \frac{1}{2} (\theta' - \theta)^T F_s (\theta' - \theta)
\end{aligned}$$

where

$$F_s = \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\nabla_\theta \log(\pi_\theta(a|s)) \nabla_\theta \log(\pi_\theta(a|s))^T \right]$$

is known as the Fisher information matrix. Removing the factor of 1/2 and plugging this into our original problem (our original goal was to not have the policies differ too much), we get the following update rule:

$$\begin{aligned} \theta' &\leftarrow \arg \max_{\theta'} (\theta' - \theta)^T \nabla_\theta J(\theta) \\ \text{subject to } &(\theta' - \theta)^T F_s (\theta' - \theta) \leq \epsilon \quad \forall s \end{aligned}$$

However, since $D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s)) \leq \epsilon \quad \forall s$ must hold at all points in the state space, it is hard to adhere to such a constraint. Instead, the TRPO and natural gradient papers approximate the constraint as

$$D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s)) \leq \epsilon \quad \forall s \approx \mathbb{E}_{s \sim p_\theta(s)} [D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s))]$$

In doing so, our approximated update rule becomes

$$\begin{aligned} \theta' &\leftarrow \arg \max_{\theta'} (\theta' - \theta)^T \nabla_\theta J(\theta) \\ \text{subject to } &(\theta' - \theta)^T F (\theta' - \theta) \leq \epsilon \end{aligned}$$

where $F = \mathbb{E}_{s \sim p_\theta(s)} [F_s]$. Note the natural gradient paper just approximates this as

$$F = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log(\pi_\theta(a|s)) \nabla_\theta \log(\pi_\theta(a|s))^T$$

where F_s is essentially approximated by a single sample estimator $\nabla_\theta \log(\pi_\theta(a|s)) \nabla_\theta \log(\pi_\theta(a|s))^T$ in each rollout and we take the average of the single sample estimators of F_s in each rollout to get a single sample of F . There are N rollouts, so we take the average of N samples of F to approximate it.

Using the derivation above for minimizing a linear approximation given a distance constraint with $c = 1$ and $A = F$, we can simplify the update rule to:

$$\begin{aligned} \theta' &= \theta - \sqrt{\frac{\epsilon}{\nabla_\theta J(\theta)^T F^{-1} \nabla_\theta J(\theta)}} F^{-1} \nabla_\theta J(\theta) \\ &= \theta - \alpha F^{-1} \nabla_\theta J(\theta) \end{aligned}$$

where $\alpha = \sqrt{\frac{\epsilon}{\nabla_\theta J(\theta)^T F^{-1} \nabla_\theta J(\theta)}}$

6 Entropy and Information Gain

6.1 Basic Definitions

Entropy for a random variable X is the expected surprise:

$$H(X) = - \sum_x p_X(x) \log(p_X(x))$$

If n is the number elements in a sequence of random variables, the probability of a typical sequence is approximately

$$\begin{aligned} & 2^{-nH(X)} \\ &= 2^{n \sum_x p_X(x) \log(p_X(x))} \\ &= \prod_x 2^{n p_X(x) \log(p_X(x))} \\ &= \prod_x (2^{\log(p_X(x))})^{n p_X(x)} \\ &= \prod_x p_X(x)^{n p_X(x)} \end{aligned}$$

Note that typical sequences essentially take up all the probability mass out of all possible sequences.

Kullback–Leibler (KL) divergence is expected excess surprise from using Q as a model instead of P when the actual distribution is P :

$$D_{KL}(y||x) = \sum_i y_i \ln \left(\frac{y_i}{x_i} \right)$$

For any given training point (a, b) , logistic loss and cross entropy loss are basically trying find model parameters that takes in a as input to produce a probability distribution x , which minimizes the KL divergence from the y , where y is a distribution that has probability 1 for the true label b . We can also think of logistic loss and cross entropy loss as trying to maximize the probability of a typical sequence (minimize the entropy H) with an arbitrary number of elements n (treat n as a constant) with respect to $x(a)$.

Joint entropy is written as

$$H(X, Y) = \sum_x \sum_y p_{X,Y}(x, y) \log \left(\frac{1}{p_{X,Y}(x, y)} \right)$$

Conditional entropy is written as

$$\begin{aligned} H(X|Y) &= \sum_y p_Y(y) \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\ &= \sum_y p_Y(y) \sum_x H(X|Y = y) \end{aligned}$$

Equivalently

$$\begin{aligned}
H(X|Y) &= \sum_y p_Y(y) \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\
&= \sum_y \sum_x p_Y(y) p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\
&= \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\
&= \mathbb{E}_{X,Y} \left[\log \left(\frac{1}{p_{X|Y}(x|y)} \right) \right]
\end{aligned}$$

Note that the notation is a bit overloaded, since $H(X|Y = y) = \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right)$, but $H(X|Y)$ is not a function of Y , but an expected value. For instance, if we define $H'(X|Y)$ as a function of Y using the first definition of $H'(X|Y = y) = \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right)$, then technically $H(X|Y = y) = \mathbb{E}_Y [H'(X|Y)]$.

We can derive the chain rule of entropy as follows

$$\begin{aligned}
H(X|Y) &= \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\
&= \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{p_Y(y)}{p_{X|Y}(x|y) p_Y(y)} \right) \\
&= \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{1}{p_{X|Y}(x|y) p_Y(y)} \right) - \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{1}{p_Y(y)} \right) \\
&= \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{1}{p_{X,Y}(x, y)} \right) - \sum_y \log \left(\frac{1}{p_Y(y)} \right) \sum_x p_{X,Y}(x, y) \\
&= H(X, Y) - \sum_y \log \left(\frac{1}{p_Y(y)} \right) p_Y(y) \\
&= H(X, Y) - H(Y)
\end{aligned}$$

We define the information gain of a random variable X obtained from an observation of a random variable Y taking on value y as

$$\begin{aligned}
IG(X, y) &= \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_X(x)} \right) - \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\
&= \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_X(x)} \right) - H(X|Y = y)
\end{aligned}$$

Equivalently

$$IG(X, y) = \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_X(x)} \right) - \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) = D_{KL}(p_{X|Y=y} || p_X)$$

Essentially, $IG(X, y)$ is the expected excess surprise from using the prior p_X as a model instead of $p_{X|Y=y}$, which is the actual model we know $Y = y$.

The mutual information or expected information gain is

$$\begin{aligned}
I(X; Y) &= \mathbb{E}_Y [IG(X, y)] \\
&= \sum_y p_Y(y) \left(\sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_X(x)} \right) - \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \right) \\
&= \sum_y p_Y(y) \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_X(x)} \right) - \sum_y p_Y(y) \sum_x p_{X|Y}(x|y) \log \left(\frac{1}{p_{X|Y}(x|y)} \right) \\
&= \sum_y \sum_x p_Y(y) p_{X|Y}(x|y) \log \left(\frac{1}{p_X(x)} \right) - H(X|Y) \\
&= \sum_y \sum_x p_{X,Y}(x, y) \log \left(\frac{1}{p_X(x)} \right) - H(X|Y) \\
&= \sum_x \log \left(\frac{1}{p_X(x)} \right) \sum_y p_{X,Y}(x, y) - H(X|Y) \\
&= \sum_x \log \left(\frac{1}{p_X(x)} \right) p_X(x) - H(X|Y) \\
&= H(X) - H(X|Y)
\end{aligned}$$

Note that mutual information is symmetric:

$$\begin{aligned}
I(X; Y) &= H(X) - H(X|Y) \\
&= H(X) - (H(X, Y) - H(Y)) \\
&= H(Y) - (H(X, Y) - H(X)) = H(Y) - H(Y|X)
\end{aligned}$$

and can also be expressed as

$$\begin{aligned}
I(X; Y) &= D_{KL}(P_{(X,Y)} || P_X \otimes P_Y) \\
&= \mathbb{E}_{X,Y} \left[\log \frac{p(x, y)}{p(x)p(y)} \right] \\
&= \mathbb{E}_{X,Y} \left[\log \frac{p(x|y)}{p(x)} \right] \\
&= \mathbb{E}_{X,Y} \left[\log \frac{p(y|x)}{p(y)} \right]
\end{aligned}$$

Considering $\mathbb{E}_{X,Y} \left[\log \frac{p(y|x)}{p(y)} \right] = H(Y) - H(Y|X)$, we can think of drawing vertical lines and seeing the entropy decrease along the points the vertical lines intersect compared to the overall distribution of Y . For instance, the red lines are the full region of support of Y which has higher entropy than the orange line, which is just one point (deterministic) in the first diagram while the second

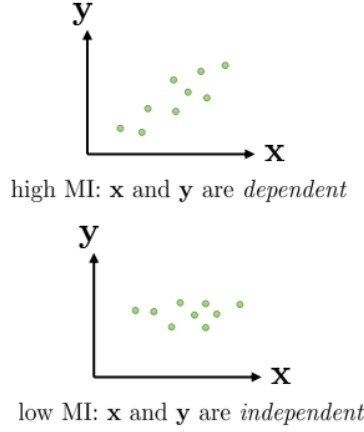


Figure 1: Mutual Information/Information Gain Visualization

diagram, the orange and red line have around the same spread. Note that even if the first diagram had a negative slope, the mutual information would be the same since x is deterministic given y and vice versa. Mutual information is 0 when X and Y are independent. In the discrete case where X and Y are deterministic given each other and uniform, the mutual information is $\log(n)$ where n is the number of values that x and y can take on since $p(x) = p(y) = p(x, y) = \frac{1}{n}$.

Note that the term "information gain" can also be overloaded since we sometimes use it to refer to the expected information gain (mutual information). For instance, we define the information gain of a split in a decision as the mutual information $I(Y, S_{i,x})$, with respect to $S_{i,x}$ where Y is a random variable representing the class of a training example in the current node and $S_{i,x} = \mathbf{1}[X_i < x]$ is an indicator random variable indicating the i th feature is smaller than x . We essentially define the information gain (which refers to mutual information in the case of decision trees) of each split here as the entropy of the parent node (the prior) minus the expected entropy of the children nodes.

6.2 Entropy Minimization Proof for Discrete Distribution

Note that when the actual distribution is P , the model Q that minimizes the entropy is P :

$$\begin{aligned} \underset{q}{\operatorname{argmin}} \quad & - \sum_i p_i \ln(q_i) \\ \text{subject to} \quad & \vec{\mathbf{1}}^T q = 1 \\ & q \geq 0 \end{aligned}$$

This is the same as

$$\begin{aligned} \underset{q}{\operatorname{argmin}} \quad & -\sum_i p_i \ln(q_i) \\ \text{subject to} \quad & \vec{\mathbf{1}}^T q - 1 = 0 \\ & -q \leq 0 \end{aligned}$$

Since the problem is convex for $p \geq 0, q \geq 0$ (Hessian of the objective with respect to q is $\operatorname{diag}(\frac{p_i}{q_i^2})$), any point (if it exists) that satisfies the KKT conditions is a solution with 0 duality gap.

$$L(q, \lambda, \nu) = -\sum_i p_i \ln(q_i) - \lambda^T q + \nu(\vec{\mathbf{1}}^T q - 1)$$

$$-q \leq 0 \quad (1)$$

$$\lambda \geq 0 \quad (2)$$

$$\vec{\mathbf{1}}^T q - 1 = 0 \quad (3)$$

$$\lambda \odot (-q) = \vec{\mathbf{0}} \quad (4)$$

$$\nabla(-\sum_i p_i \ln(q_i)) - \lambda^T \nabla(q) + \nu \nabla(\vec{\mathbf{1}}^T q - 1) = 0 \quad (5)$$

Starting with condition (5):

$$\begin{aligned} \nabla(-\sum_i p_i \ln(q_i)) - \lambda^T \nabla(q) + \nu \nabla(\vec{\mathbf{1}}^T q - 1) &= 0 \\ \implies -p \oslash q - \lambda + \nu \vec{\mathbf{1}} &= 0 \\ \implies \lambda = \nu \vec{\mathbf{1}} - p \oslash q \end{aligned}$$

Note that technically $p > 0$ and $q > 0$, otherwise any 0 element should technically not be part of the vector since the distribution does not include that element. Plugging this into condition (4):

$$\begin{aligned} \lambda \odot (-q) &= \vec{\mathbf{0}} \\ \implies \lambda \odot q &= \vec{\mathbf{0}} \\ \implies (\nu \vec{\mathbf{1}} - p \oslash q) \odot q &= \vec{\mathbf{0}} \\ \implies \nu q - p &= \vec{\mathbf{0}} \\ \implies \nu q &= p \\ \implies \nu q^T \mathbf{1} &= p^T \mathbf{1} \\ \implies \nu &= 1 \end{aligned}$$

Since $\nu = 1$

$$\nu q = p \implies q = p$$

First note that $\ln a \leq a - 1$ for all $a > 0$.

We will now show that $-D_{KL}(p||q) \leq 0$ which means that $D_{KL}(p||q) \geq 0$

$$\begin{aligned}
 -D(p||q) &= -\sum_x p(x) \ln \frac{p(x)}{q(x)} \\
 &= \sum_x p(x) \ln \frac{q(x)}{p(x)} \\
 &\stackrel{(a)}{\leq} \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \\
 &= \sum_x q(x) - \sum_x p(x) \\
 &= 1 - 1 \\
 &= 0
 \end{aligned}$$

Figure 2: Enter Caption

This is a proof that holds for continuous distributions (we can replace sum with integral) since proving KL divergence is nonnegative means the entropy is always larger if we use a distribution $Q \neq P$ as a model compared to using the P as the model when the real distribution is P .

7 Miscellaneous Notes

Adding an affine transformation of x to $h(x)$ does not change the Bregman divergence with respect to h :

$$\begin{aligned}\tilde{h}(x) &= h(x) + a^T x + b \\ D_{\tilde{h}}(y||x) &= \tilde{h}(y) - (\tilde{h}(x) + (y-x)^T \nabla \tilde{h}(x)) \\ &= h(y) + a^T y + b - (h(x) + a^T x + b + (y-x)^T \nabla (h(x) + a^T x + b)) \\ &= h(y) + a^T y - h(x) - a^T x - (y-x)^T \nabla h(x) - (y-x)^T a \\ &= (h(y) - h(x) - (y-x)^T \nabla h(x)) + a^T y - a^T x - (y-x)^T a \\ &= D_h(y||x)\end{aligned}$$

3. Temporal Difference Learning

See <https://web.stanford.edu/group/pdplab/pdphandbook/handbookch10.html>.

Note with the TD learning in <https://inst.eecs.berkeley.edu/~cs188/sp23/assets/notes/cs188-sp23-note13.pdf>, we can think of $f(s)$ as a one-hot encoding, where each state corresponds to one element and use the actual values of the states as the parameters w .